پلتفرم مرغ‌یار — مدیریت پرورش مرغ گوشتی

راهنمای اجرا

قدم به قدم: از نصب تا استقرار

API، داشبورد وب، اپ موبایل آفلاین‌فرست و سرویس تحلیل — با خروجی مورد انتظار هر قدم و عیب‌یابی

نسخه سند:	همگام با مخزن (شاخه `arena/9419963c-farm01`)
پشته فنی:	`NestJS` . `PostgreSQL` . `Redis` . `React` . `Expo` . `FastAPI`
پیش‌نیازها:	`Docker` . `Python 3.10+` . `Node.js 20+` (اختیاری)
داده نمونه:	`owner@farm.ir / Secret1234`

فهرست

گام ۰۱ پیش‌نیازها را بررسی کنید 3

گام ۰۲ کد را بگیرید و وابستگی‌ها را نصب کنید 3

گام ۰۳ بسته دامنه را بسازید (پیش‌نیاز پنهان) 4

گام ۰۴ پایگاه داده را بسازید (PGlite، بدون نصب چیزی) 4

گام ۰۵ داده نمونه را بریزید 5

گام ۰۶ API را روشن کنید 5

گام ۰۷ سلامت API و ورود را آزمون کنید 6

گام ۰۸ داشبورد وب را روشن کنید 7

گام ۰۹ سرویس تحلیل را روشن کنید (اختیاری ولی ارزشمند) 7

گام ۱۰ رفتار بدون سرویس تحلیل را بیازمایید 8

گام ۱۱ اپ موبایل را اجرا کنید 8

گام ۱۲ آفلاین بودن موبایل را بیازمایید 9

گام ۱۳ آزمون‌ها و کیفیت کد را اجرا کنید 9

گام ۱۴ مستندات را بسازید 10

گام ۱۵ نسخه PDF همین راهنما را بسازید 11

گام ۱۶ با Docker Compose مستقر کنید 11

گام ۱۷ چک‌لیست نهایی پیش از دمو را بزنید 12

گام ۱۸ خاموش‌کردن و تمیزکاری 12

پیوست الف — نقشه اجزا و پورت‌ها 12

پیوست ب — متغیرهای محیطی 13

پیوست ج — عیب‌یابی: خطا، علت، درمان 13

پیوست د — دستورهایی که در CI اجرا می‌شوند 14

این سند، مسیر کامل بالا آوردن سامانه از **صفر** است. ترتیب گام‌ها مهم است: هر گام به گام قبلی تکیه دارد. برای همین همه‌چیز شماره‌گذاری شده است:

- **گام 1 تا 5:** آماده‌سازی — نصب، بسته دامنه، پایگاه داده.
- **گام 6 تا 8:** بالا آوردن API و داشبورد و آزمون ورود.
- **گام 9 تا 12:** سرویس تحلیل و اپ موبایل و آزمون آفلاین.
- **گام 13 تا 15:** آزمون‌ها، مستندات و همین PDF.
- **گام 16 تا 18:** استقرار با Docker، چک‌لیست نهایی، خاموش‌کردن.

هر گام سه بخش دارد: **چه کاری** (توضیح یک‌خطی)، **دستور** (چیزی که تایپ می‌کنید)، و **نقطه بررسی** (چطور مطمئن شوید درست انجام شد). اگر جایی خطا گرفتید، جدول عیب‌یابی در پیوست ج را ببینید.

> همه دستورها از پوشه platform/ اجرا می‌شوند، مگر آن‌که خلافش نوشته شده باشد.

گام 1. پیش‌نیازها را بررسی کنید

چه کاری: مطمئن شوید Node، npm و Python با نسخه درست نصب‌اند.

اول نسخه‌ها را ببینید:

```
node -v && npm -v && python3 --version
```

خروجی واقعی محیط آزموده‌شده:

```
v22.22.3
10.9.8
Python 3.11.2
```

نقطه بررسی: هر سه عدد چاپ شدند. اگر node -v کمتر از 20 بود، ادامه ندهید؛ NestJS 11 و Vite 6 روی نسخه‌های قدیمی بالا نمی‌آیند. Docker فقط برای گام 16 لازم است و اگر نصب نباشد، بقیه راهنما کار می‌کند.

ابزار	نسخه لازم	بررسی
Node.js	20 یا بالاتر (با 22 آزموده شده)	node -v
npm	10 یا بالاتر	npm -v
Python	3.10 یا بالاتر (با 3.11 آزموده شده)	python3 --version
Docker	اختیاری، فقط برای استقرار	docker compose version

گام 2. کد را بگیرید و وابستگی‌ها را نصب کنید

چه کاری: مخزن را کلون کنید و بسته‌ها را دقیقاً مطابق package-lock.json نصب کنید.

1. مخزن را بگیرید و به پوشه پلتفرم بروید:

```
git clone https://github.com/you/Farm01.git Farm01
cd Farm01/platform
```

2. نصب را اجرا کنید:

```
npm ci
```

`npm ci` را به‌جای `npm install` بزنید تا **دقیقاً** همان نسخه‌های قفل‌شده نصب شوند؛ نتیجه روی ماشین شما و `CI` یکی می‌شود.

3. اگر `npm ci` در دسترس نبود:

```
npm install --no-audit --no-fund
```

خروجی واقعی: نصب کامل در حدود 23 **ثانیه** (926 بسته، شامل ورک‌اسپیس‌های `apps/*` و `packages/*`) .

نقطه بررسی:

```
ls node_modules/.bin | grep -c vite
ls node_modules/vazirmatn/fonts/ttf/Vazirmatn-Regular.ttf
```

اولی باید عددی 1 یا بیشتر بدهد، دومی باید مسیر فونت را چاپ کند. اگر فونت پیدا نشد، گزارش‌های `PDF` فارسی بی‌ریخت می‌شوند.

گام 3. بسته دامنه را بسازید (پیش‌نیاز پنهان)

چه کاری: `@farm/domain` را از سورس به `dist` ببرید.

`API` و وب، بسته دامنه را از **خروجی ساخته‌شده** می‌خوانند، نه از سورس. اگر این گام را رد کنید، در گام 6 با خطای `Cannot find module '@farm/domain'` روبه‌رو می‌شوید:

```
npm run build:domain
```

خروجی واقعی: حدود 2 **ثانیه** ، بی‌پیام.

نقطه بررسی:

```
ls packages/domain/dist/index.js && node -e
"console.log(Object.keys(require('./packages/domain/dist')).length, 'export')"
```

گام 4. پایگاه داده را بسازید (PGlite، بدون نصب چیزی)

چه کاری: جدول‌ها را با مهاجرت‌ها ایجاد کنید.

`PGlite` همان `PostgreSQL` است که به `WASM` کامپایل شده؛ همان `numeric` ، همان `timestamptz` ، همان ایندکس یکتا و `ON CONFLICT` . فقط یک نویسنده درون‌فرایند دارد و برای تولید مناسب نیست — برای تولید، گام 16 را ببینید.

1. مسیر داده را تعیین کنید (روی دیسک، تا با ری‌استارت پاک نشود) :

```
export PGLITE_DATA_DIR=./.pgdata
```

2. مهاجرت‌ها را اجرا کنید:

```
npm run db:migrate
```

خروجی واقعی:

```
[Database] PGlite (./.pgdata) — مناسب توسعه و تست
مهاجرت‌های اجراشده: 0001_init.sql, 0002_model_v2.sql, 0003_operations.sql, 0004_alert_thresholds.sql
```

3. اگر خواستید مطمئن شوید تکرارپذیر است، همان دستور را بار دوم بزنید:

```
npm run db:migrate
```

این بار می‌گوید «مهاجرت جدیدی نبود.» — مهاجرت‌ها در جدول `migrations_` ثبت می‌شوند و اجرای دوباره چیزی را خراب نمی‌کند.

اگر واقعی می‌خواهید: به‌جای این گام، `docker compose up -d postgres` را بزنید و این‌ها را صادر کنید: `PG_DRIVER=postgres` و `DATABASE_URL=postgres://farm:farm@localhost:5432/farm` ، بعد همان `npm run db:migrate` را اجرا کنید.

گام 5. داده نمونه را بریزید

چه کاری: یک مزرعه، دو سالن و یک گله فعال با 28 روز ثبت بسازید.

```
npm run db:seed
```

خروجی واقعی:

```
[Seed] داده نمونه ساخته شد: owner@farm.ir / Secret1234 امروز(2026-10-24)
```

نقطه بررسی: تعداد جدول‌های دامنه را بشمارید:

```
npm run docs:schema && grep -c "CREATE TABLE" docs/architecture/schema.sql
```

اگر خواستید از صفر شروع کنید:

```
rm -rf .pgdata
npm run db:migrate && npm run db:seed
```

گام 6. API را روشن کنید

چه کاری: سرور را روی پورت 4000 بالا بیاورید (ترمینال 1) .

1. پورت را تنظیم کنید:

```
export PORT=4000
```

2. حالت توسعه (ری‌استارت خودکار) را اجرا کنید:

```
npm run dev:api
```

خروجی واقعی (خلاصه لاگ بالا آمدن) :

```
[Database] PGlite (./.pgdata) — مناسب توسعه و تست
[NestApplication] Nest application successfully started
[Bootstrap] API روی http://0.0.0.0:4000/api/v1 درایور: (pglite)
```

اگر خواستید مثل تولید اجرا کنید، به‌جای حالت توسعه:

```
npm run build:api
PORT=4000 node apps/api/dist/main.js
```

نقطه بررسی: ترمینال را باز نگذارید; یک ترمینال دیگر باز کنید و گام 7 را بزنید.

گام 7. سلامت API و ورود را آزمون کنید

چه کاری: سه آزمون کوتاه که نشان می‌دهد API زنده، پایگاه داده وصل و ورود کار می‌کند.

1. سلامت:

```
curl -s http://127.0.0.1:4000/api/v1/health
```

```
{"status":"ok","service":"farm-api","time":"2026-10-24T05:19:24.192Z"}
```

2. ورود و گرفتن توکن:

```
TOKEN=$(curl -s -X POST http://127.0.0.1:4000/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"owner@farm.ir","password":"Secret1234"}' | python3 -c "import
sys,json;print(json.load(sys.stdin)['accessToken'])")
echo "${TOKEN:0:20}…"
curl -s -H "Authorization: Bearer $TOKEN" http://127.0.0.1:4000/api/v1/farm
```

3. سند تعاملی API را در مرورگر باز کنید: `http://localhost:4000/api/docs`

(نسخه JSON خام: `/api/docs-json`) .

4. نگهبان نرخ را ببینید (شش تلاش پشت‌سرهم با رمز غلط) :

```
for i in 1 2 3 4 5 6; do
curl -s -o /dev/null -w "%{http_code} " -X POST http://127.0.0.1:4000/api/v1/auth/login \
-H "Content-Type: application/json" -d '{"email":"ghost@farm.ir","password":"WrongPass1"}'
done; echo
```

خروجی واقعی: `401 401 401 401 401 429` — تلاش ششم با سقف نرخ بسته می‌شود و هدر `Retry-After` برمی‌گردد.

برای آزمون دستی بعدی، سرور را ری‌استارت کنید یا `RATE_LIMIT_DISABLED=1` بدهید.

گام 8. داشبورد وب را روشن کنید

چه کاری: رابط کاربری را روی پورت `5173` بالا بیاورید (ترمینال 2) .

```
cd Farm01/platform
VITE_API_TARGET=http://127.0.0.1:4000 npm run dev:web
```

خروجی واقعی:

```
VITE v6.4.5 ready in 144 ms
 Local: http://localhost:5173/
```

1. مرورگر را باز کنید: `http://localhost:5173`
2. با `owner@farm.ir` و رمز `Secret1234` وارد شوید.
3. در داشبورد باید چهار `KPI` و سه نمودار با داده واقعی ببینید.

نکته‌ای که باید بدانید: مرورگر **هرگز** به `localhost:4000` درخواست نمی‌زند؛ همه مسیرها نسبی (`/api/v1`) هستند و `Vite` آن‌ها را پروکسی می‌کند. به همین دلیل نه `CORS` لازم است، نه آدرس `API` داخل باندل سخت می‌شود. اگر `API` روی پورت دیگری است، همان را در `VITE_API_TARGET` بگذارید.

گام 9. سرویس تحلیل را روشن کنید (اختیاری ولی ارزشمند)

چه کاری: سرویس `FastAPI` را روی پورت `8000` بالا بیاورید (ترمینال 3) .

1. محیط پایتون را بسازید و بسته‌ها را نصب کنید:

```
cd services/ai
python3 -m venv .venv
./.venv/bin/pip install -r requirements.txt
```

2. سرویس را اجرا کنید:

```
./.venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

3. سلامت آن را ببینید:

```
curl -s http://127.0.0.1:8000/health
```

```
{"status":"ok","service":"farm-ai","version":"0.2.0",
"models":["gompertz-growth","breed-curves","disease-rules","early-warning-trends",
"loading-optimizer","feed-forecast","speech-to-text(off)"]}
```

4. `API` را با آدرس این سرویس اجرا کنید: `AI_URL=http://127.0.0.1:8000` (گام 6) .

نقطه بررسی — `API` واقعاً از سرویس استفاده می‌کند:

```
curl -s -H "Authorization: Bearer $TOKEN" http://127.0.0.1:4000/api/v1/ai/status
```


```
{"enabled":true,"available":true,"url":"http://127.0.0.1:8000","latencyMs":39,
"models":["gompertz-growth", "..."],"checkedAt":"2026-10-24T05:25:07.092Z"}
```


و پیش‌بینی یک گله:

```
curl -s -H "Authorization: Bearer $TOKEN" \
http://127.0.0.1:4000/api/v1/ai/flocks/<flockId>/forecast | head -c 200
```


```
{"source":"ai","fallback":false,"projectedWeightGr":2306,"targetWeightGr":2450,"confidence":0.95,"
method":"gompertz-fit"}
```

گام 10. رفتار بدون سرویس تحلیل را بیازمایید

چه کاری: سرویس را خاموش کنید و ببینید سیستم نمی‌شکند.

1. در ترمینال سرویس تحلیل `Ctrl+C` بزنید.
2. همان درخواست پیش‌بینی گام 9 را دوباره بزنید.
3. پاسخ باید `source: "local"` و `fallback: true` بدهد و **هیچ خطایی** به کاربر نشان داده نشود.

این تفاوت «هوش مصنوعی به‌عنوان بهبود» با «هوش مصنوعی به‌عنوان تکیه‌گاه» است: نبودِ سرویس، داده‌های ثبت‌شده را هرگز قفل نمی‌کند.

گام 11. اپ موبایل را اجرا کنید

چه کاری: سرور Expo را بالا بیاورید و اپ را روی گوشی یا شبیه‌ساز باز کنید.

1. اگر بسته دامنه را نساخته‌اید، بسازید:

```
npm run build:domain
```

2. نوع‌ها را بدون نیاز به گوشی بررسی کنید:

```
npm run typecheck:mobile
```

3. سرور Expo را اجرا کنید:

```
npm run start:mobile
```

4. یکی از این سه راه را انتخاب کنید:

- گوشی واقعی: اپ Expo Go را نصب کنید و QR را اسکن کنید.
- اندروید امولاتور: کلید `a` در ترمینال Expo.

- کلید `i` : (فقط `macOS`) `iOS`

5. آدرس `API` را در `apps/mobile/app.json` تنظیم کنید:

```
"extra": { "apiUrl": "http://192.168.1.20:4000", "aiUrl": "http://192.168.1.20:8000" }
```

`localhost` روی گوشی به خودِ گوشی اشاره می‌کند، نه رایانه؛ `IP` رایانه در شبکه محلی را بگذارید و مطمئن شوید هر دو روی یک `Wi-Fi` هستند.

گام 12. آفلاین بودن موبایل را بیازمایید

چه کاری: مهم‌ترین ویژگی این اپ را با چهار حرکت ساده بسنجید.

1. وارد شوید و یک ثبت روزانه بزنید؛ باید بدون اینترنت هم ذخیره شود.
2. `Wi-Fi` یا داده را قطع کنید و چند ثبت دیگر بزنید؛ هیچ دکمه‌ای نباید غیرفعال شود.
3. اینترنت را وصل کنید؛ صف به‌طور خودکار ارسال می‌شود.
4. همان ثبت را دوباره بفرستید (سناریوی قطع وسط ارسال)؛ پاسخ باید `"status":"duplicate"` باشد، نه یک ردیف دوم.

آزمون سمت سرور، بدون گوشی:

```
curl -s -X POST http://127.0.0.1:4000/api/v1/sync/push \
-H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d
'{"operations":[{"clientId":"test-op-0001","type":"daily_log","flockId":"<flockId>","payload":{"da
te":"2026-10-20","mortality":3,"feedKg":120}}]}'
```

ارسال دوباره‌ی همان بدنه، `duplicate` می‌دهد.

گام 13. آزمون‌ها و کیفیت کد را اجرا کنید

چه کاری: چهار فرمان، چهار لایه بررسی.

1. نوع‌ها:

```
npm run typecheck
```

2. قواعد امنیتی (هش رمز، اعتبارسنجی ورودی، RBAC) روی درخت کد:

```
npm run check:security
```

3. آزمون‌ها:

```
npm test
```

4. آزمون سرویس تحلیل:

```
npm run test:ai
```

خروجی واقعی:

```
قواعد امنیتی برقرار است — 17 کنترلر، 74 مسیر، 2 استثنای مستند.
# tests 99 # pass 99 # fail 0
# tests 114 # pass 114 # fail 0
Tests 22 passed (22)
23 passed
```

آزمون API روی PGlite **در حافظه** اجرا می‌شود؛ پس به پایگاه داده بیرونی نیاز ندارد و در CI هم سبز است.

برای این‌که ببینید نگهبان امنیتی واقعاً کار می‌کند (نه فقط سبز است) ، یک @Roles را از یک مسیر بردارید: npm run check:security می‌شکند و نام همان مسیر را چاپ می‌کند.

گام 14. مستندات را بسازید

چه کاری: DDL، مشخصات OpenAPI و یک فایل HTML خودبسنده تولید کنید.

1. DDL را از پایگاه داده بگیرید:

```
npm run docs:schema
```

2. مشخصات API را از سرور در حال اجرا بگیرید (API باید روشن باشد) :

```
API_URL=http://127.0.0.1:4000 npm run docs:openapi
```

3. فایل HTML یکجا را بسازید:

```
npm run docs:html
```

خروجی واقعی:

```
جدول‌ها: 16 · خطوط: 130
openapi.yaml 29 schema نوشته شد — 54 مسیر،
farm-architecture.html نوشته شد — 560 کیلوبایت
```

فایل	چیست
docs/architecture/README.md	سند معماری هشت‌بخشی (دستی نوشته می‌شود)
docs/architecture/schema.sql	DDL واقعی، تولیدشده
docs/architecture/openapi.yaml	مشخصات API، تولیدشده از سرور در حال اجرا
docs/architecture/farm-architecture.html	همه‌ی بالا در یک فایل، با فونت و تصویر درون‌فایل

گام 15. نسخه PDF همین راهنما را بسازید

چه کاری: از همین سند یک PDF فارسی و راست‌به‌چپ بگیرید.

```
npm run docs:pdf
```

خروجی واقعی:

```
نوشته شد — 14 صفحه، 84 کیلوبایت RUNBOOK.pdf
```

فایل در `docs/RUNBOOK.pdf` است. اگر فونت شکسته چاپ شد، یعنی بسته `vazirmatn` نصب نیست؛ گام 2 را تکرار کنید.

گام 16. با Docker Compose مستقر کنید

چه کاری: همه اجزا را با `Postgres` و `Redis` واقعی بالا بیاورید.

1. به پوشه پلتفرم بروید و دو متغیر الزامی را صادر کنید:

```
cd platform
export JWT_SECRET=$(openssl rand -hex 32)
export CORS_ORIGINS=https://app.example.com
```

2. سرویس‌ها را بسازید و اجرا کنید:

```
docker compose up -d --build
```

3. وضعیت را ببینید:

```
docker compose ps
```

ترتیب بالا آمدن: `postgres` (با `healthcheck`) ← `redis` ← `api` (مهاجرت‌ها را خودکار اجرا می‌کند) ← `ai`.

4. سلامت و لاگ را ببینید:

```
curl -s http://127.0.0.1:4000/api/v1/health
docker compose logs -f api
```

قفل ایمنی تولید: اگر `JWT_SECRET` نمونه یا کوتاه باشد، یا `CORS_ORIGINS` باز باشد، برنامه به‌جای بالا آمدن ناامن، همان لحظه می‌شکند:

```
پیکربندی ناامن برای تولید: Error:
- JWT_SECRET تنظیم نشده یا کوتاه است (حداقل 32 نویسه).
- CORS_ORIGINS در تولید باید فهرست دقیق دامنه‌ها باشد، نه خالی و نه *.
```

این پیام خطا نیست که دور بزنید؛ چک‌لیست تنظیمات است.

گام 17. چک‌لیست نهایی پیش از دمو را بزنید

چه کاری: هفت بررسی سریع، پیش از نشان دادن سیستم.

- ☐ `curl http://127.0.0.1:4000/api/v1/health` پاسخ `{"status":"ok",...}` می‌دهد
- ☐ ورود `owner@farm.ir / Secret1234` در `http://localhost:5173` انجام می‌شود
- ☐ داشبورد چهار `KPI` و سه نمودار با داده واقعی نشان می‌دهد
- ☐ `http://localhost:4000/api/docs` باز می‌شود
- ☐ `npm test` سبز است (99 + 114 + 22)
- ☐ `npm run check:security` سبز است
- ☐ با خاموش‌بودن سرویس تحلیل، پیش‌بینی همچنان جواب می‌دهد (`fallback: true`)

گام 18. خاموش‌کردن و تمیزکاری

چه کاری: سرورها را ببندید و اگر لازم شد، داده توسعه را پاک کنید.

1. سرورهای محلی را ببندید:

```
lsof -ti:4000,5173,8000 | xargs -r kill
```

2. داده توسعه را پاک کنید:

```
rm -rf .pgdata
```

3. خروجی‌های ساخت را پاک کنید:

```
rm -rf apps/api/dist apps/api/dist-test packages/domain/dist apps/web/dist
```

4. اگر `Docker` استفاده کردید:

```
docker compose down # داده‌ها می‌مانند
docker compose down -v # داده‌ها هم پاک می‌شوند
```

پیوست الف — نقشه اجزا و پورت‌ها

جزء	پوشه	پورت پیش‌فرض	نقش
بسته دامنه	`packages/domain`	—	محاسبات مشترک (FCR، هشدار، پیش‌بینی محلی) · **پیش‌نیاز بقیه**
API	`apps/api`	`4000`	`PostgreSQL/PGlite`، JWT/RBAC، NestJS
داشبورد وب	`apps/web`	`5173`	`React + Vite; /api` را به API پروکسی می‌کند
اپ موبایل	`apps/mobile`	`8081` (Expo)	`Expo/React Native`، آفلاین‌فرست

جزء	پوشه	پورت پیش‌فرض	نقش
سرویس تحلیل	`services/ai`	`8000`	`FastAPI`؛ اختیاری، نبودش `API` را نمی‌شکند

نکته معماری: **فقط `API` به پایگاه داده دسترسی دارد.** وب و موبایل هیچ اتصال مستقیمی ندارند؛ وب از مسیر نسبی `/api/v1` (پروکسی `Vite`) و موبایل از `expo.extra.apiUrl` استفاده می‌کند.

پیوست ب — متغیرهای محیطی

یک فایل `env.` از نمونه بسازید:

```
cp .env.example .env
```

برای توسعه محلی تقریباً همه پیش‌فرض‌ها درست‌اند. فقط این‌ها اهمیت دارند:

متغیر	پیش‌فرض	چه‌وقت تغییر می‌دهید
`PORT`	`4000`	وقتی `4000` اشغال است
`PG_DRIVER`	`postgres` در `.env.example`	برای توسعه بدون `PGLITE_DATA_DIR`، Postgres را تنظیم کنید
`DATABASE_URL`	`postgres://farm:farm@localhost:5432/farm`	فقط با `Postgres` واقعی
`PGLITE_DATA_DIR`	`memory://`	`./.pgdata`. برای داده پایدار روی دیسک
`JWT_SECRET`	نمونه	**در تولید الزامی**؛ تولید با مقدار نمونه بالا نمی‌آید
`CORS_ORIGINS`	`http://localhost:3000`	آدرس داشبورد؛ چند مقدار با کاما
`REDIS_URL`	`redis://localhost:6379`	برای شمارنده نرخ مشترک؛ نبودش خطا نیست
`AI_URL`	`http://localhost:8000`	آدرس سرویس تحلیل
`RATE_LIMIT_GLOBAL_MAX`	`1200`	سقف کلی هر نشانی در دقیقه

`env.` خودکار خوانده نمی‌شود؛ ساده‌ترین راه، صادرکردن متغیرها در همان ترمینال است (همان کاری که در گام‌ها کردیم) یا استفاده از `Docker Compose` (گام 16).

پیوست ج — عیب‌یابی: خطا، علت، درمان

نشانه	علت	راه‌حل
`Cannot find module '@farm/domain'`	بسته دامنه ساخته نشده	`npm run build:domain`
`EADDRINUSE: 4000`	نمونه‌ی قبلی `API` زنده است	`lsof -ti:4000 | xargs kill` یا `PORT=4001`
صفحه وب سفید، کنسول خطای `404`	پروکسی به `API` اشتباه اشاره می‌کند	`VITE_API_TARGET` را درست کنید

نشانه	علت	راه‌حل
`SQLITE_BUSY` یا قفل `PGlite`	دو فرایند روی یک پوشه `PGLITE_DATA_DIR`	یکی را ببندید یا پوشه جدا بدهید
پیکربندی ناامن برای تولید	`NODE_ENV=production` با تنظیمات نمونه	`JWT_SECRET` حداقل `32` نویسه و `CORS_ORIGINS` دقیق
`[ Redis ]` `Redis` در دسترس نیست	`Redis` بالا نیست (هشدار، نه خطا)	`docker compose up -d redis` یا نادیده بگیرید
ورود سریع با `429`	سقف نرخ فعال شد	`5` دقیقه صبر یا `RATE_LIMIT_DISABLED=1` در آزمون دستی
گزارش `PDF` با حروف شکسته	فونت وزیرمتن نصب یا بارگیری نشده	`npm ci` و وجود `node_modules/vazirmatn`
همیشه `source: "local"`	`AI_URL` اشتباه یا سرویس خاموش	سرویس تحلیل را بالا بیاورید (گام `9`) — این رفتار خطا نیست
موبایل «خطای شبکه» می‌دهد	`apiUrl` روی `localhost` مانده	`IP` رایانه در `app.json`
آزمون `API` روی `CI` قرمز	بسته دامنه ساخته نشده	در `CI` هم `npm run build:domain` پیش از تست اجرا می‌شود

پیوست د — دستورهایی که در CI اجرا می‌شوند

`.github/workflows/platform-ci.yml` دقیقاً همین ترتیب را روی هر `Pull Request` اجرا می‌کند؛ پس اگر گام‌های بالا روی ماشین شما سبز باشند، `CI` هم سبز است:

1. `npm ci`
2. `npm run build:domain`
3. `npm run typecheck`
4. `npm run typecheck:mobile`
5. `npm run check:security`
6. `npm test`
7. `npm run build:web`
8. `python -m pytest -q` در `services/ai`